Лабораторная работа №16

Тема работы: «Разработка программ создания, корректировки и просмотра типизированных (бинарных) файлов»

1 Цель работы

Сформировать умение разрабатывать и реализовывать программы создания, корректировки и просмотра пользовательских бинарных файлов

2 Задание

Разработайте программу ведения типизированного файла, содержащего некоторые данные (свой вариант). В программе должны быть реализованы следующие функции:

– ввод записей;

– корректировка записей;

– удаление записей;

– просмотр записей на экране.

Требуемая функция выбирается с помощью меню.

12. Сведения об автомобилях:

- марка;
- страна производитель;
- год выпуска;
- цена.

Выведите информацию об автомобилях выпуска прошлого года в порядке убывания цен.

3 Оснащение

ПК, задание по варианту, среда разработки Visual Studio.

4 Порядок выполнения работы

Код программы:

```
#include <iostream>
#include <fstream>
#include <string>
#include <ctime>
using namespace std;

struct Car {
    char brand[50];
    char country[50];
    int year;
    double price;
};

const int MAX = 100;

Car inputCar() {
    Car c;
    cout << "Марка: ";
    cin >> c.brand;
```

```
20.     cout << "Страна: ";
21.     cin >> c.country;
22.     cout << "Год выпуска: ";
23.     cin >> c.year;
24.     cout << "Цена: ";
25.     cin >> c.price;
26.     return c;
27. }
28.
29.
30. void saveToFile(Car cars[], int count, const string& filename) {
31.     ofstream fout(filename, ios::binary | ios::trunc);
32.     if (!fout) {
33.         cout << "Ошибка открытия файла для записи!\n";
34.         return;
35.     }
36.     fout.write((char*)&count, sizeof(count));
37.     fout.write((char*)cars, sizeof(Car) * count);
38.     fout.close();
39. }
40.
41. int loadFromFile(Car cars[], const string& filename) {
42.     ifstream fin(filename, ios::binary);
43.     if (!fin) return 0;
44.     int count;
45.     fin.read((char*)&count, sizeof(count));
46.     fin.read((char*)cars, sizeof(Car) * count);
47.     fin.close();
48.     return count;
49. }
50.
51. void viewCars(Car cars[], int count) {
52.     cout << "\nСписок автомобилей:\n";
53.     for (int i = 0; i < count; i++) {
54.         cout << i + 1 << ". " << cars[i].brand << " | "<< cars[i].country << " | "<<
cars[i].year << " | " << cars[i].price << endl;
55.     }
56. }
57.
58. void deleteCar(Car cars[], int& count) {
59.     viewCars(cars, count);
60.     cout << "Введите номер для удаления: ";
61.     int idx; cin >> idx;
62.     if (idx < 1 or idx > count) return;
63.     for (int i = idx - 1; i < count - 1; i++) cars[i] = cars[i + 1];
64.     count--;
65. }
66.
67. void editCar(Car cars[], int count) {
68.     viewCars(cars, count);
69.     cout << "Введите номер для редактирования: ";
70.     int idx; cin >> idx;
71.     if (idx < 1 || idx > count) return;
72.     cars[idx - 1] = inputCar();
73. }
74.
75. void sortByPrice(Car cars[], int count) {
76.     for (int i = 0; i < count - 1; i++) {
77.         for (int j = 0; j < count - i - 1; j++) {
78.             if (cars[j].price < cars[j + 1].price) {
79.                 Car tmp = cars[j];
80.                 cars[j] = cars[j + 1];
81.                 cars[j + 1] = tmp;
82.             }
```

```
 83.         }
 84.     }
 85. }
 86.
 87. void showLastYearCars(Car cars[], int count) {
 88.     time_t t = time(nullptr);
 89.     tm now;
 90.     localtime_s(&now, &t);
 91.     int lastYear = now.tm_year + 1900 - 1;
 92.
 93.     Car filtered[MAX];
 94.     int fCount = 0;
 95.     for (int i = 0; i < count; i++) {
 96.         if (cars[i].year == lastYear) {
 97.             filtered[fCount++] = cars[i];
 98.         }
 99.     }
100.
101.     sortByPrice(filtered, fCount);
102.
103.     cout << "\nАвтомобили " << lastYear << " года:\n";
104.     for (int i = 0; i < fCount; i++) {
105.         cout << filtered[i].brand << " | " << filtered[i].country << " | " << fil-
tered[i].year << " | " << filtered[i].price << endl;
106.     }
107. }
108.
109. int main() {
110.     setlocale(LC_ALL, "Russian");
111.     string filename = "cars.dat";
112.     Car cars[MAX];
113.     int count = loadFromFile(cars, filename);
114.
115.     int choice;
116.     do {
117.         cout << "\nМеню:\n"
118.             << "1. Добавить запись\n"
119.             << "2. Редактировать запись\n"
120.             << "3. Удалить запись\n"
121.             << "4. Просмотреть все\n"
122.             << "5. Автомобили прошлого года\n"
123.             << "0. Выход\n"
124.             << "Ваш выбор: ";
125.         cin >> choice;
126.
127.         switch (choice) {
128.         case 1: if (count < MAX) cars[count++] = inputCar(); break;
129.         case 2: editCar(cars, count); break;
130.         case 3: deleteCar(cars, count); break;
131.         case 4: viewCars(cars, count); break;
132.         case 5: showLastYearCars(cars, count); break;
133.         case 0: saveToFile(cars, count, filename); break;
134.         }
135.     } while (choice != 0);
136.
137.     return 0;
138. }
```

```
1. Добавить запись
2. Редактировать запись
3. Удалить запись
4. Просмотреть все
5. Автомобили прошлого года
0. Выход
Ваш выбор: 4

Список автомобилей:
1. БМВ | Гермния | 2025 | 324546
2. BMW | Germany | 2025 | 50000
3. Mercedes | Germany | 2025 | 45000
4. Audi | Germany | 2024 | 40000

Меню:
1. Добавить запись
2. Редактировать запись
3. Удалить запись
4. Просмотреть все
5. Автомобили прошлого года
0. Выход
Ваш выбор: 5

Автомобили 2025 года:
БМВ | Гермния | 2025 | 324546
BMW | Germany | 2025 | 50000
Mercedes | Germany | 2025 | 45000
```

Рисунок 1 - Результат выполнения программы

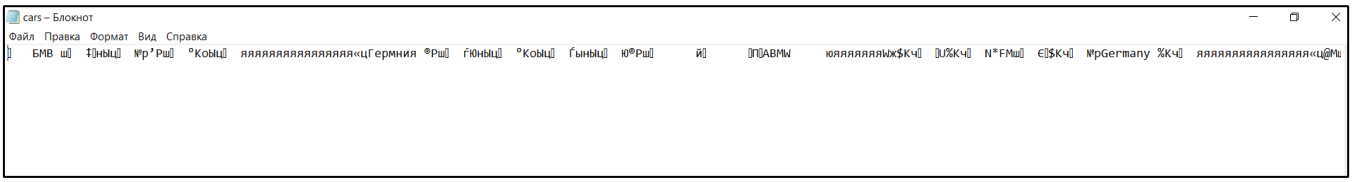

Рисунок 2 - Результат записи в файл

5. Контрольные вопросы и задания

1. Дайте краткое понятие типизированных (бинарных) файлов и их роль в программировании на C++

Типизированные (бинарные) файлы — это файлы, в которых данные хранятся в двоичном формате, то есть в виде последовательности байтов, полностью соответствующих внутреннему представлению данных в памяти компьютера. Их роль в программировании на C++ заключается в том, что они позволяют сохранять и восстанавливать сложные структуры данных (например, массивы, структуры, объекты) без преобразования в текстовый вид, что обеспечивает высокую скорость работы и экономию памяти.

2. Раскройте значение операций создания, записи и чтения данных из типизированных файлов и почему они важны для работы с данными

– Создание файла позволяет подготовить пространство для хранения данных.

– Запись обеспечивает сохранение информации в двоичном виде, что делает возможным точное восстановление данных без потерь.

– Чтение позволяет загружать данные обратно в программу в исходном виде.

Эти операции важны, так как они дают возможность работать с большими объемами информации, сохранять состояние программы между запусками и эффективно обмениваться данными между различными приложениями.

3. Сформулируйте сущность структуры данных в контексте бинарных файлов и как они организованы в памяти компьютера

Структура данных в контексте бинарных файлов — это способ организации информации в памяти, который напрямую переносится в файл. В памяти данные располагаются в виде последовательности байтов, где каждый элемент структуры имеет фиксированное место и размер. При записи в бинарный файл эта последовательность сохраняется без изменений, что позволяет при чтении восстановить структуру в точности такой же, как она была в памяти.

4. Изложите особенности работы с указателями при записи и чтении данных из типизированных файлов на C++ и как это отличается от работы с текстовыми файлами

При работе с бинарными файлами указатели используются для передачи адреса данных в функции записи (write) и чтения (read). Запись и чтение происходят напрямую по адресу памяти, что позволяет сохранять и восстанавливать данные в их исходном виде. В отличие от текстовых файлов, где данные преобразуются в символы и строки, бинарные файлы работают с «сырыми» байтами, поэтому указатели играют ключевую роль в точной передаче информации.

5. Раскройте особенности использования маркеров при чтении и записи данных в бинарные файлы и как они помогают обеспечить корректную работу программы

Маркеры (позиционные указатели) определяют текущее положение в файле, откуда будет происходить чтение или запись. Они помогают:

- управлять порядком доступа к данным;
- перемещаться к нужному месту в файле;
- избегать ошибок при последовательной обработке информации.

Таким образом, маркеры обеспечивают корректность работы программы, позволяя точно контролировать, какие данные читаются или записываются.

6. Дайте определение функций seekg() и seekp() в контексте работы с типизированными файлами на C++ и их роль в управлении

- seekg() — функция, которая управляет положением маркера чтения (get pointer) в бинарном файле. Она позволяет перемещаться к нужной позиции для последующего чтения.
- seekp() — функция, которая управляет положением маркера записи (put pointer). Она используется для указания места, куда будут записаны данные.

Обе функции играют важную роль в управлении доступом к файлу, так как позволяют работать не только с последовательным, но и с произвольным доступом к данным.